

```

#include <systemc.h>

SC_MODULE(ProtocolMonitor) {
    sc_in<bool> clk;
    sc_in<bool> req;
    sc_in<bool> ack;

    enum State { WAIT_FOR_REQ, WAIT_FOR_ACK };
    State next_state;

    // Configuration variables
    const sc_time m_clk_period;
    const int m_timeout_cycles;
    const sc_time m_watchdog_limit;

    // Timestamp tracker
    sc_time expected_timeout_time;

    void monitor_method() {
        switch (next_state) {
            case WAIT_FOR_REQ:
                if (req.read() == true) {
                    cout << "[Monitor] REQ at " << sc_time_stamp() << ". Setting watchdog timer." << endl;

                    // Calculate exactly when the simulator will force us to wake up
                    expected_timeout_time = sc_time_stamp() + m_watchdog_limit;

                    next_state = WAIT_FOR_ACK;

                    // Wait for ACK, but force wake-up if m_watchdog_limit passes first
                    next_trigger(m_watchdog_limit, ack.value_changed_event());
                    // Instead of value_changed_event(), wait ONLY for a rising edge on ACK
                    next_trigger(m_watchdog_limit, ack.posedge_event());
                }
            } else {

```

```

        next_trigger(clk.pos());
    }
    break;

case WAIT_FOR_ACK:
    // Timestamp Check Logic: Did we wake up because the clock/watchdog expired?
    // TO-DO: what if timeout and ack both happen at exactly the same time?
    if (sc_time_stamp() >= expected_timeout_time && ack.read() == false) {
        SC_REPORT_ERROR("Protocol", "Watchdog expired! Upstream hardware hung.");
        next_state = WAIT_FOR_REQ;
        next_trigger(clk.pos());
    }
    // We woke up before or exactly at the timeout limit, check if ACK is valid
    else if (ack.read() == true) {
        cout << "[Monitor] Transaction success at " << sc_time_stamp() << endl;
        next_state = WAIT_FOR_REQ;
        next_trigger(clk.pos());
    }
    // False alarm (e.g., ACK toggled low instead of high). Keep waiting with remaining time.
    else {
        sc_time remaining_time = expected_timeout_time - sc_time_stamp();
        next_trigger(remaining_time, ack.value_changed_event());
        // Guaranteed to be a real ACK assertion because we filtered out falling edges!
        cout << "[Monitor] Transaction success at " << sc_time_stamp() << endl;
        next_state = WAIT_FOR_REQ;
        next_trigger(clk.pos());
    }
    break;
}

}

// Constructor using initializer list to scale the timeout window safely
ProtocolMonitor(sc_module_name name, sc_time clk_period, int timeout_cycles)
: sc_module(name),
  m_clk_period(clk_period),

```

```

    m_timeout_cycles(timeout_cycles),
    m_watchdog_limit(clk_period * timeout_cycles) // Safe, scaled timeout limit
{
    SC_METHOD(monitor_method);
    next_state = WAIT_FOR_REQ;
    next_trigger(SC_ZERO_TIME);
}
};

```

can pass several types of arguments to `next_trigger()`, matching the dynamic needs:

- **Single Event:** `next_trigger(my_event);`
- **Port Event:** `next_trigger(clk.pos());` or `next_trigger(req.value_changed_event());`
- **Time Delay:** `next_trigger(10, SC_NS);`
- **Logical OR (Any event):** `next_trigger(event_A | event_B);`
- **Logical AND (All events):** `next_trigger(event_A & event_B);`

When evaluating **simulation performance**, an `SC_METHOD` using `next_trigger()` is almost always **significantly faster and consumes less memory** than an `SC_THREAD` using `wait()`.

How to Choose (The Trade-Off)

While `SC_METHOD` wins on raw performance, it sacrifices **code readability** for complex chronological behavior.

- **Use `SC_METHOD` + `next_trigger()` when:** writing underlying library blocks, clocks, bus hubs, simple state machines, register files, or hardware behaviors that can easily be written as a flat `switch/case` block
- **Use `SC_THREAD` + `wait()` when:** modeling complex procedural software, testbench stimulus sequences, or multi-step networking protocols (forcing these behaviors into a state machine inside an `SC_METHOD` results in unreadable, hard-to-maintain "spaghetti code")

Assumption made: An `ack` signal that is already high *before* or *at the exact moment* a `req` is asserted indicates a **pre-assertion protocol violation** (often caused by a stuck-at-1 fault or a slave device failing to clear its previous cycle state).

Key elements added below:

- **State Isolation via Struct:** Telemetry is gathered inside a separate `ProtocolTelemetry` datatype. This allows verification infrastructure to easily extract metric packets, export data blocks to a file, or pass metrics upward to a top-level regression manager.
- **Pre-Assertion Intercept:** Inside `WAIT_FOR_REQ`, a nested condition intercepts the state processing immediately when `req == true`. If `ack` matches `true`, an explicit `SC_REPORT_ERROR` flags a structural interface bug, logs the fault telemetry, and bypasses the watchdog logic entirely.
- **Dynamic String Formatting:** Custom error payloads leverage `sc_time_stamp().to_string()` directly in the `SC_REPORT_ERROR` stream macros, ensuring tracking software receives absolute, machine-readable simulation time metadata upon any exception.

```
#include <systemc.h>
#include <iostream>
#include <string>

// Production Best Practice: Use a dedicated telemetry/logging structure
struct ProtocolTelemetry {
    uint64_t total_transactions = 0;
    uint64_t timeout_errors    = 0;
    uint64_t pre_assert_errors = 0;

    void print_summary(const std::string& name) const {
        std::cout << "\n=== TELEMETRY REPORT [" << name << "] ===\n"
                  << "  Total Successful Transactions: " << total_transactions << "\n"
                  << "  Timeout Violations:                " << timeout_errors    << "\n"
                  << "  Pre-Assert Violations (ACK):          " << pre_assert_errors << "\n"
                  << "===== \n" << std::endl;
    }
};

SC_MODULE(ProtocolMonitor) {
public:
    sc_in<bool> clk;
    sc_in<bool> req;
    sc_in<bool> ack;
```

```

// Public telemetry object for testbench inspection
ProtocolTelemetry telemetry;

private:
enum State { WAIT_FOR_REQ, WAIT_FOR_ACK };
State m_next_state;

// Read-only configuration variables initialized once
const sc_time m_clk_period;
const int m_timeout_cycles;
const sc_time m_watchdog_limit;

// Cycle tracking state
sc_time m_expected_timeout_time;

void monitor_method() {
    switch (m_next_state) {
        case WAIT_FOR_REQ:
            if (req.read() == true) {
                // TELEMETRY: Detect violation if ACK is already high before/on REQ
                if (ack.read() == true) {
                    SC_REPORT_ERROR("PROTOCOL_VIOLATION",
                        ("ACK is already HIGH at the assertion of REQ at " +
                         sc_time_stamp().to_string()).c_str());

                    telemetry.pre_assert_errors++;

                    // Remain in WAIT_FOR_REQ state and wait for next clock cycle
                    next_trigger(clk.pos());
                    break;
                }

                // Normal behavior: Request active, ACK is cleanly low
                m_expected_timeout_time = sc_time_stamp() + m_watchdog_limit;
                m_next_state = WAIT_FOR_ACK;
            }
    }
}

```

```

        next_trigger(m_watchdog_limit, ack.posedge_event());
    } else {
        next_trigger(clk.pos());
    }
    break;

case WAIT_FOR_ACK:
    // Check if the scheduler woke us due to the watchdog timeout expiration
    if (sc_time_stamp() >= m_expected_timeout_time && ack.read() == false) {
        SC_REPORT_ERROR("PROTOCOL_TIMEOUT",
            ("Watchdog expired! No ACK received within " +
             std::to_string(m_timeout_cycles) + " cycles at " +
             sc_time_stamp().to_string()).c_str());

        telemetry.timeout_errors++;
        m_next_state = WAIT_FOR_REQ;
        next_trigger(clk.pos());
    }
    // Clean wakeup: Valid rising edge on ACK caught before timeout
    else if (ack.read() == true) {
        telemetry.total_transactions++;
        m_next_state = WAIT_FOR_REQ;
        next_trigger(clk.pos());
    } // else NOT needed as both watchdog/timeout and ack cases covered above
    break;
}
}

```

```

public:
    // Safe modern C++ constructor using explicit type handling and initialization lists
    ProtocolMonitor(sc_module_name name, sc_time clk_period, int timeout_cycles)
        : sc_module(name),
          m_next_state(WAIT_FOR_REQ),
          m_clk_period(clk_period),

```

```

        m_timeout_cycles(timeout_cycles),
        m_watchdog_limit(clk_period * timeout_cycles),
        m_expected_timeout_time(SC_ZERO_TIME)
    {
        SC_METHOD(monitor_method);
    }
};

// =====
// TESTBENCH SUITE
// =====
int sc_main(int argc, char* argv[]) {
    // Disable intrusive default SystemC warning banners for clean production logs
    sc_report_handler::set_actions(SC_INFO, SC_DO_NOTHING);

    sc_clock clk("clk", 10, SC_NS); // 10ns clock period
    sc_signal<bool> req("req");
    sc_signal<bool> ack("ack");

    // Instantiate monitor with a strict 4-cycle timeout threshold (40ns)
    ProtocolMonitor monitor("monitor", sc_time(10, SC_NS), 4);
    monitor.clk(clk);
    monitor.req(req);
    monitor.ack(ack);

    // Initialize bus lines
    req.write(false);
    ack.write(false);

    // --- CASE 1: Standard Success Cycle ---
    sc_start(15, SC_NS);
    req.write(true); // REQ asserted at 15ns
    sc_start(10, SC_NS);
    ack.write(true); // ACK asserted at 25ns (Well within 40ns window)
    sc_start(10, SC_NS);

```

```

req.write(false);
ack.write(false);

// --- CASE 2: Watchdog Timeout Cycle ---
sc_start(20, SC_NS);
req.write(true); // REQ asserted at 65ns
sc_start(50, SC_NS); // Let 50ns pass (Exceeds our 40ns watchdog window)
req.write(false);

// --- CASE 3: Telemetry Pre-Assertion Violation Cycle ---
sc_start(20, SC_NS);
ack.write(true); // Crucial Fault: Slave line gets stuck high early!
sc_start(10, SC_NS);
req.write(true); // Master asserts REQ while ACK is illegally active
sc_start(10, SC_NS);
req.write(false);
ack.write(false);

// Finalize simulation runtime frame
sc_start(20, SC_NS);

// Output telemetry report for verification validation
monitor.telemetry.print_summary(monitor.name());

return 0;
}

```

Even if static sensitivity list is used above, the code is not cleaner anyway, due to:

Any call to `next_trigger(event)` completely clears out the previous sensitivity rules for that process.

The scheduler sets a temporary execution pointer. Once that dynamic event (`ack`) triggers and the method runs to completion, **the sensitivity list is completely empty.**

- If the code reaches a path where it exits the function without an explicit instruction, the process becomes **orphaned**.

- It will never execute again for the entire remainder of the simulation, even if the clock transitions a million times.

To return to the constructor's static list, we must explicitly tell the scheduler to reload it by typing `next_trigger();` with no arguments. Because we have to write a line of code either way, writing the explicit event (`clk.pos()`) is widely considered safer and more self-documenting than relying on an empty `next_trigger();` call.

However, to optimize performance, classify processes into two categories:

1. Proactive Processes (Do NOT use `dont_initialize()`) [1]

These processes actively generate stimulus or manage time right out of the gate.

- **Examples:** Clock generators, testbench stimulus drivers, reset sequencers.
- **Why:** They must run at time zero to kick off the simulation.

2. Reactive Processes (STRATEGICALLY use `dont_initialize()`)

These processes do nothing but sit around and wait for an external trigger to occur. They do not drive signals at startup; they only consume them.

- **Examples:** Protocol monitors (like our `ProtocolMonitor`), bus checkers, pipeline registers, memory slaves.
- **Why:** Running them at time zero is completely useless because no transactions have started yet.